TP ACDC80 - INVESTIGATION DFIR & PIPELINE DEVSECOPS

Analyse Forensique de Dump GPS Embarqué & Automatisation CI/CD de la Chaîne de Preuve

CONFIDENTIEL OCLCC | MASTER 1 CYBERSÉCURITÉ | AFFAIRE GHOST DELACROIX

Contexte Forensic	Saisie d'un boîtier GPS sur le véhicule de la cible « Ghost Delacroix ». Nettoyage de dump mémoire corrompu.
Format TP	Travaux Pratiques. Le script Python est à compléter (zones `# TODO`).
Livrables Obligatoires	`input_log.txt`, `parse_gps.py`, `output_cleaned.csv`, Dépôt GitHub (Commits signés SSH), `.github/workflows/main.yml`.

1. Objectifs du Travail Pratique

Dans ce TP, vous incarnez un expert en réponse à incident (DFIR). Vous avez récupéré un fichier de logs bruts (`input_log.txt`) extrait de la mémoire d'un traceur GPS artisanal. Le fichier est partiellement corrompu et contient du bruit matériel.

Votre mission : développer un script Python (`parse_gps.py`) capable de :

1. Lire le fichier en ignorant les erreurs d'encodage matériel.
2. Extraire de façon robuste les coordonnées géographiques, même si certaines lignes sont illisibles.
3. Nettoyer (valeurs NaN) et dédoublonner les données.
4. Générer un fichier CSV propre accompagné de son empreinte cryptographique (SHA-256).

Le tout devra être versionné via Git avec des commits signés par clé SSH, et audité par une Pipeline d'Intégration Continue (CI) sur GitHub Actions.

2. Étape 0 : Setup Git, Clés SSH & Commits Signés

Objectif : Configurer un environnement d'investigation non-répudiable dans lequel chaque modification apportée au script d'analyse est signée numériquement.

1. Générez votre clé SSH de signature et configurez Git :

```
ssh-keygen -t ed25519 -C "expert.dfir@oclcc.gouv.fr" -f ~/.ssh/id_ed25519_dfir
git config --global user.name "Votre Nom (Expert DFIR)"
git config --global user.email "votre.email@student.com"
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519_dfir.pub
git config --global commit.gpgsign true
```

2. Initialisez le dépôt et créez le fichier de preuve brut `input_log.txt` à la racine du projet :

```
[2026-10-24 14:00:12.123456] SYSTEM_START Booting firmware v4.1
[2026-10-24 14:01:05.999111] GNSS_ACCESS_LIB Lat 48.8584 / Lon 2.2945 / Alt 35.2
[2026-10-24 14:01:05.999111] GNSS_ACCESS_LIB: Lat 48.8584/Lon 2.2945 / Alt 35.2
[2026-10-24 14:02:10.000000] GNSS_ACCESS_LIB: Lat nan / Lon 2.3000 / Alt 40.0
[2026-10-24 14:03:15.456789] Trip Data Lat 48.860012 / Lon 2.310045 / Alt 38.1
[2026-10-24 14:04:20.100200] GNSS_ACCESS_LIB: Lat 49.1234/ Lon corrupt_val / Alt 10.0
[2026-10-24 14:05:00.000000] GNSS ACCESS_LIB: Lat 48.8590/ Lon 2.2950 / Alt 35.0
[2026-10-24 14:06:00.000000] SYSTEM_STOP LOW battery shutdown
```

3. Étape 1 : Lecture Tolérante & Filtrage des Trames

Objectif : Ouvrir le dump sans provoquer d'erreur due à l'encodage matériel, et filtrer les lignes utiles au tracking.

Consignes : Le fichier est encodé en latin-1. Vous devez ne conserver que les lignes contenant la chaîne `"GNSS_ACCESS_LIB"` OU la chaîne `"Trip Data"`. Complétez le fichier `parse_gps.py`.

```
import sys

def lire_et_filtrer_fichier(input_file):
    lines_filtrees = []
    # TODO: Ouvrir le fichier avec l'encodage 'latin-1'

        # TODO: Parcourir le fichier ligne par ligne

            # TODO: Si la ligne contient "GNSS_ACCESS_LIB" ou "Trip Data",
            # l'ajouter à lines_filtrees (pensez à faire .strip())

    return lines_filtrees

if __name__ == "__main__":
    lignes = lire_et_filtrer_fichier("input_log.txt")
    print(f"[ETAPE 1] Lignes retenues : {len(lignes)}")
    # Sortie Attendue: [ETAPE 1] Lignes retenues: 6
```

> **Action Git Signée - Étape 1**
> `git add parse_gps.py && git commit -S -m "feat(parser): ajout lecture latin-1 et filtrage trames GPS"`

4. Étape 2 : Parsing Avancé & Gestion des Exceptions

Objectif : Extraire les composants de chaque trame (horodatage, lat, lon, alt). Si une ligne est corrompue (ex: `corrupt_val`), le script doit l'ignorer au lieu de crasher.

Consignes : Extraire l'horodatage en retirant les microsecondes. Séparer les coordonnées (`/`), les convertir en float. Protéger cette conversion avec un bloc `try / except`.

```
def extraire_donnees_brutes(lines):
    records = []
    for line in lines:
        # TODO: Mettre en place un bloc try / except (ValueError, IndexError)

            # TODO: Extraire le timestamp situé entre crochets [...] et retirer les
microsecondes.

            # TODO: Découper la fin de la ligne avec les séparateurs ':' ou '-' puis
par '/'

            # TODO: Extraire lat, lon, alt et les convertir en float().

            # TODO: Ajouter le tuple (timestamp, lat, lon, alt) à la liste records
            pass
```

5. Étape 3 : Nettoyage Mathématique & Dédoublonnage

Objectif : Traiter le bruit capteur en éliminant les NaN, et en supprimant les points géographiques redondants (le véhicule est resté à l'arrêt).

Consignes : Utilisez `math.isnan()` pour écarter les valeurs impossibles. Arrondissez à 2 décimales. Si le dernier point validé est à moins de +/- 1.0° de latitude ET longitude, ignorez le point actuel.

```
import math

def nettoyer_et_dedoublonner(records):
    cleaned = []
    for ts, lat, lon, alt in records:
        # TODO: Vérifier si lat, lon, ou alt sont des NaN (math.isnan). Si oui, ignorer.

        # TODO: Arrondir lat, lon, alt à 2 décimales avec round()

        # TODO: Dedoublonnage spatial (+/- 1.0 degré) par rapport au dernier élément de 'cleaned'

        # TODO: Ajouter le point valide (ts, lat_r, lon_r, alt_r) dans cleaned
        pass

    return cleaned
    # Test & Sortie Attendue: [ETAPE 3] Points nettoyés et dédoublonnés: 2
```

> **Action Git Signée - Étape 3**
> `git add parse_gps.py && git commit -S -m "refactor(cleaner): filtrage NaN et dedoublonnage géographique"`

6. Étape 4 : Exportation CSV & Scellé SHA-256

Objectif : Formater la preuve au format standard CSV et produire une empreinte cryptographique (hash) scellant l'intégrité de votre fichier de sortie.

```
import csv
import hashlib

def exporter_et_hasher(cleaned_records, output_file="output_cleaned.csv"):
    # TODO: Ouvrir output_file en mode écriture ('w'), newline='', encoding='utf-8'

    # TODO: Utiliser csv.writer pour écrire l'en-tête ["timestamp", "latitude", "longitude", "altitude"]

    # TODO: Parcourir cleaned_records et écrire chaque ligne dans le CSV

    # TODO: Calcul de l'empreinte SHA-256 en lisant output_file en binaire ('rb') avec hashlib
    sha256_hash = "" # Mettre votre hash final ici (hexdigest)

    print(f"SHA-256: {sha256_hash}")
    print(f"FLAG: FLAG_OCLCC_{sha256_hash[:12]}")
```

7. Étape 5 : Workflow DevSecOps CI/CD & Flag CTF

Objectif : Mettre en place la validation automatisée du code Python et l'audit de la chaîne de preuve sur un serveur d'Intégration Continue (GitHub Actions).

Créez le fichier `.github/workflows/main.yml` à la racine du dépôt :

```
name: DFIR GPS Parser CI Workflow
on:
  push:
    branches: [ "main" ]

jobs:
  verify-forensic-parser:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout du code source
        uses: actions/checkout@v3

      - name: Configuration Python 3.10
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Exécution du script forensic
        run: python parse_gps.py

      - name: Vérification du fichier CSV de sortie
        run: test -f output_cleaned.csv || (echo "[ERROR] CSV manquant" && exit 1)

      - name: Contrôle d'intégrité SHA-256 en CI
        run: sha256sum output_cleaned.csv
```

Comprendre la Pipeline CI/CD (Explication DevSecOps)

La démarche DevSecOps repose sur le concept de *Shift Left* (déplacer les contrôles de sécurité et de qualité au plus tôt dans la chaîne de développement) :

- **Isolation et Stérilité (ubuntu-latest) :** L'exécution locale sur la machine de l'investigateur peut souffrir de biais. La CI installe une machine Linux vierge et stérile pour garantir la reproductibilité judiciaire.
- **Non-Répudiation des Commits :** Grâce aux clés SSH configurées à l'Étape 0, chaque modification poussée déclenchant le workflow est signée et attribuable de manière certaine.
- **Contrôle de Réussite Automatisé :** L'étape `test -f output_cleaned.csv` garantit que le traitement s'est exécuté sans crasher. Si le script lève une exception, le workflow passe au rouge et bloque le livrable.

> **Action Git Signée Finale - Push vers GitHub**
>
> `git add .github/ && git commit -S -m "ci: ajout workflow de validation automatisée DevSecOps" && git push origin main`
>
> **VALIDATION FINALE DU TP : Vérifiez que l'onglet 'Actions' sur GitHub est GREEN**

8. Corrigé Enseignant (Réservé au formateur)

Le code complet et fonctionnel pour le fichier `parse_gps.py` :

```
import sys
import math
import csv
import hashlib

def traiter_dump_gps(input_file, output_file):
    # --- ÉTAPE 1: Lecture tolérante latin-1 & Filtrage ---
    with open(input_file, 'r', encoding='latin-1') as f:
        raw_lines = [line.strip() for line in f if "GNSS_ACCESS_LIB" in line or "Trip
Data" in line]

    # --- ÉTAPE 2 & 3: Parsing, Nettoyage NaN et Dédoublonnage ---
    cleaned = []
    for line in raw_lines:
        try:
            # Isolation horodatage sans microsecondes
            ts = line.split(']')[0].replace('[', '').split('.')[0]

            # Extraction et conversion float
            parts = line.split(':')[-1] if ':' in line else line.split('Data')[-1]
            items = parts.split('/')

            lat = float(items[0].replace('Lat', '').strip())
            lon = float(items[1].replace('Lon', '').strip())
            alt = float(items[2].replace('Alt', '').strip())

            # Rejet des NaN
            if math.isnan(lat) or math.isnan(lon) or math.isnan(alt):
                continue

            lat_r, lon_r, alt_r = round(lat, 2), round(lon, 2), round(alt, 2)

            # Dédoublonnage spatial (+/- 1.0 degré)
            if cleaned:
                last_lat = cleaned[-1][1]
                last_lon = cleaned[-1][2]
                if abs(lat_r - last_lat) <= 1.0 and abs(lon_r - last_lon) <= 1.0:
                    continue

            cleaned.append((ts, lat_r, lon_r, alt_r))

        except (ValueError, IndexError):
            continue

    # --- ÉTAPE 4: Export CSV et Calcul SHA-256 ---
    with open(output_file, 'w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerow(["timestamp", "latitude", "longitude", "altitude"])
        writer.writerows(cleaned)

    hasher = hashlib.sha256()
    with open(output_file, 'rb') as f:
```